UNIVERSITÉ SAINT JEAN
SAINT JEAN INGÉNIEUR

# CAHIER DE RÉALISATION
# Conteneurisation et gestion des workloads avec Magnum

Rédigé par:
- ESSONO EFFA Etienne Florian
- NGNINTEDEM FEUPI Vaillant

**Sous la supervision : M. NGUIMBUS Emmanuel**

**Année Académique 2025 - 2026**

# Plan

INTRODUCTION

I. PRÉSENTATION DE L’ENVIRONNEMENT LOGICIEL ET DES TECHNOLOGIES

I.1 – ENVIRONNEMENT LOGICIEL

I.2 – PRÉSENTATION DES OUTILS

II. PRÉSENTATION DE L’ARCHITECTURE

1. Architecture logique

2. Architecture physique

CONCLUSION

# INTRODUCTION

Le dossier de réalisation formalise l'étape de déploiement et de configuration technique de la solution. Après avoir défini les besoins et conçu l'architecture théorique, il s'agit ici de décrire l'environnement de travail réel et la manière dont les composants **OpenStack Magnum** et **Heat** ont été instanciés. Cette phase valide la faisabilité du projet par la mise en place d'un prototype fonctionnel sur une plateforme **DevStack**.

# I. PRÉSENTATION DE L'ENVIRONNEMENT LOGICIEL ET DES TECHNOLOGIES

## I.1 – ENVIRONNEMENT LOGICIEL

Pour garantir la stabilité et la compatibilité des services OpenStack, l'environnement suivant a été sélectionné :

- **Système d'Exploitation :** Ubuntu 22.04 LTS. C'est le standard recommandé pour les déploiements DevStack en raison de sa gestion robuste des paquets et des dépendances Python.
- **Plateforme de déploiement : DevStack**. Ce choix permet de regrouper l'ensemble des contrôleurs et des services sur une configuration optimisée, facilitant les tests d'intégration entre Magnum et Heat dans un environnement contrôlé.
- **Hyperviseur de base : VMWare WorkStation** pour l'installation du SE et **KVM (Kernel-based Virtual Machine)** utilisé par Nova pour la création des instances qui serviront de nœuds de clusters.

# I. PRÉSENTATION DE L’ENVIRONNEMENT LOGICIEL ET DES TECHNOLOGIES

## I.2 – PRÉSENTATION DES OUTILS

La réalisation s'appuie sur une suite d'outils spécialisés :

- **OpenStack Magnum :** Le service de gestion de clusters (COE). Il permet de créer les "ClusterTemplates" et de piloter le cycle de vie des moteurs comme Kubernetes.
- **OpenStack Heat :** Le moteur d'orchestration. Il interprète les fichiers YAML (templates HOT) pour déployer de manière atomique l'infrastructure (VM, réseaux, stockage).
- **OpenStack CLI :** L'interface en ligne de commande utilisée pour automatiser les tâches et interagir directement avec les APIs.
- **Fichiers YAML / HOT :** Le langage utilisé pour définir les infrastructures hybrides (UC02).
- **Images Cloud :** Utilisation d'images **Fedora CoreOS** ou **CentOS**, pré-configurées avec les agents nécessaires pour rejoindre un cluster Kubernetes.

# II. PRÉSENTATION DE L’ARCHITECTURE

## 1. Architecture logique

L'architecture logique décrit le flux d'informations et les dépendances entre les services logiciels.

- **L'utilisateur** interagit avec l'API de Magnum.
- **Magnum** génère une requête vers **Heat**.
- **Heat** décompose cette requête en ressources individuelles pour **Nova** (calcul), **Neutron** (réseau) et **Cinder** (stockage).
- L'authentification et les quotas sont gérés de manière transverse par **Keystone**.

Ce qui peut être assimilé ou correspondre au modèle **MVC (Model View Controler)**

# II. PRÉSENTATION DE L'ARCHITECTURE

## Le Modèle MVC (Modèle-Vue-Contrôleur)

L'architecture logique d'un service OpenStack (comme Magnum ou Heat) est très proche du modèle **MVC**. Bien qu'OpenStack soit une architecture pilotée par les événements et les APIs, l'analogie fonctionne parfaitement :

- **Le Contrôleur (API) :** Les services `magnum-api` ou `heat-api` jouent ce rôle. Ils reçoivent les requêtes de l'utilisateur (via REST), les valident et les transmettent à la logique métier.
- **Le Modèle (Database / State) :** C'est la base de données (MariaDB) où Magnum et Heat stockent l'état des clusters, des stacks et des ressources. Elle représente la "source de vérité" du système.
- **La Vue (Infrastructure / Resources) :** Dans le contexte du Cloud, la "Vue" n'est pas seulement un écran, c'est le **résultat concret** produit. Pour l'utilisateur, la "vue" de son système, c'est le cluster Kubernetes opérationnel ou les VMs créées. On peut aussi considérer le Dashboard **Horizon** comme la vue graphique.

# II. PRÉSENTATION DE L’ARCHITECTURE

## 2. Architecture physique

Dans le cadre de cette réalisation sur DevStack, l'architecture physique est simplifiée pour tenir sur un environnement de test ("All-in-One") :

- **Nœud Contrôleur & Calcul :** Une machine virtuelle unique avec 16 Go de RAM et 8 vCPUs hébergeant l'intégralité des services OpenStack.
- **Topologie Réseau :**
  - Un réseau **Management** (interne aux services).
  - Un réseau **Public/Externe** pour l'accès aux APIs.
  - Un réseau **Tenant** (isolé) où sont déployées les instances Kubernetes et les VMs de la stack hybride.

Ce qui correspond à une architecture **3-Tiers (3/3)** (Client > Contrôleur > Infrastructure/Données), typique des environnements client-serveur modernes.

# II. PRÉSENTATION DE L’ARCHITECTURE

## Le Modèle 3-Tiers (3/3) ou N-Tiers

Pour l'architecture physique, votre système correspond à une architecture **3-Tiers (3/3)**, qui peut s'étendre en **N-Tiers** selon la complexité :

- **Tier 1 : Couche de Présentation (Client) :** C'est le poste de l'administrateur avec le CLI OpenStack ou le navigateur web (Horizon). C'est là que les commandes sont initiées.
- **Tier 2 : Couche Application / Traitement (Logic) :** Ce sont les serveurs de contrôle (Controller Nodes). Ils hébergent l'intelligence : Magnum Conductor, Heat Engine, Keystone. C'est ici que les décisions d'orchestration sont prises.
- **Tier 3 : Couche de Données et d'Exécution (Data & Infrastructure) :**
  - **Données :** La base de données SQL stockant les configurations.
  - **Exécution :** Les nœuds de calcul (Compute Nodes / Nova) où les VMs et les conteneurs tournent réellement.

**Pourquoi on dit souvent N-Tiers dans le Cloud ?** Parce que dans OpenStack, la couche "Application" est elle-même divisée en des dizaines de micro-services qui discutent entre eux. On dépasse le simple schéma 3-tiers pour une architecture distribuée plus complexe.

# CONCLUSION

La phase de réalisation ainsi achevée, il était question pour nous de présenter les différents outils utilisés pour la réalisation de notre système ainsi que l’architecture sur laquelle est basé notre système. Ainsi, nous avons alors pu présenter quelle est le modèle physique de donnée qui découle de notre diagramme de classe ainsi que le modèle physique de données qui donne un peu plus d’informations concernant la structure de celui-ci.